Ramrao Adik Institute of Technology

Department of Computer Engineering

SE Mini Project-I | Abstract Presentation

NEXUSPLAY: AI Powered Game Hub

Football Manager | Epic Battle Simulation | Minecraft AI Builder

Presented by:

Anish Choudhury (24CS1004)
Atharva Inamdar (25CS1029)
Abhishek More (24CS1045)
Aryan Patil (24CS1055)

Guide:

Dr. Vishakha Gaikwad

Presentation Outline

1	**Introduction:** Overview of all three games and shared theme
2	**Football Manager:** Problem · Objectives · Architecture · Tech Stack · AI Features
3	**Epic Battle Simulation:** Problem · Objectives · Architecture · Tech Stack · AI Features
4	**Minecraft AI Builder:** Problem · Objectives · Architecture · ML Pipeline · Tech Stack · Structures
5	**Comparative Overview:** Side-by-side analysis of all three games
6	**Advantages:** Per-game strengths and shared advantages
7	**Limitations & Future Scope:** Current constraints and roadmap
8	**Conclusion:** Key takeaways and statistics
9	**References & Presenter Division:** Citations and slide ownership

Introduction — All Three Games at a Glance

⚽ Football Manager

Core Idea

Web-based football management sim where you recruit, train, and deploy squads against AI-rated opponents.

Tech

Python · Flask · SQLAlchemy · SQLite · Jinja2

Goal

Simulate realistic match outcomes using player attribute-based probability models.

⚔️ Epic Battle Simulation

Core Idea

Real-time 3D top-down battlefield where a Q-learning AI adapts its army composition each round.

Tech

Python · Panda3D · Q-Learning · Spatial Grids · NumPy

Goal

Train an adaptive opponent AI that improves strategy over multiple battles.

⛏️ Minecraft AI Builder

Core Idea

Type a natural language command; ML classifiers predict intent/size/material and build structures live in 3D.

Tech

Python · Panda3D · scikit-learn · TF-IDF · Logistic Regression

Goal

Bridge NLP and 3D game worlds with self-trained, offline classifiers (>95% accuracy).

D Y PATIL DEEMED TO BE UNIVERSITY RAMRAO ADIK INSTITUTE OF TECHNOLOGY NAVI MUMBAI

Football Manager — Problem Statement

Gap in accessible FM games: Most football manager games (FM, FIFA Career) are complex, expensive, or closed-source.

No lightweight web-based solution: No open-source browser-based manager game with real player data and realistic AI match simulation exists.

Static match engines: Existing indie FM clones use fixed win-rates; they ignore individual player attributes entirely.

Transfer & morale neglected: Simplified games skip transfer market economics, player morale, and contract negotiations.

Data not leveraged: Real league data (5 leagues, 100+ clubs) is publicly available but rarely used in academic game projects.

D Y PATIL DEEMED TO BE UNIVERSITY RAMRAO ADIK INSTITUTE OF TECHNOLOGY NAVI MUMBAI

Football Manager — Objectives

1. Build a full-stack football management game using Python Flask with a web-based UI.
2. Model player attributes (Overall, Pace, Shooting, Passing, Dribbling, Defending, Physical) and morale.
3. Design a probabilistic match engine where squad strength directly drives goal probability.
4. Implement a transfer market system with release clauses, wage negotiation, and contract tracking.
5. Support 5 real leagues: Premier League, La Liga, Bundesliga, Serie A, Ligue 1 with authentic clubs.
6. Provide a tactics setup screen (formation, mentality, pressing) that affects match outcomes.
7. Enable season simulation with league tables, fixtures, and morale updates after results.

D Y PATIL DEEMED TO BE UNIVERSITY — RAMRAO ADIK — INSTITUTE OF TECHNOLOGY NAVI MUMBAI

Football Manager — System Architecture & Match Flow

SYSTEM LAYERS

Flask Web App
Routes, Jinja2 Templates

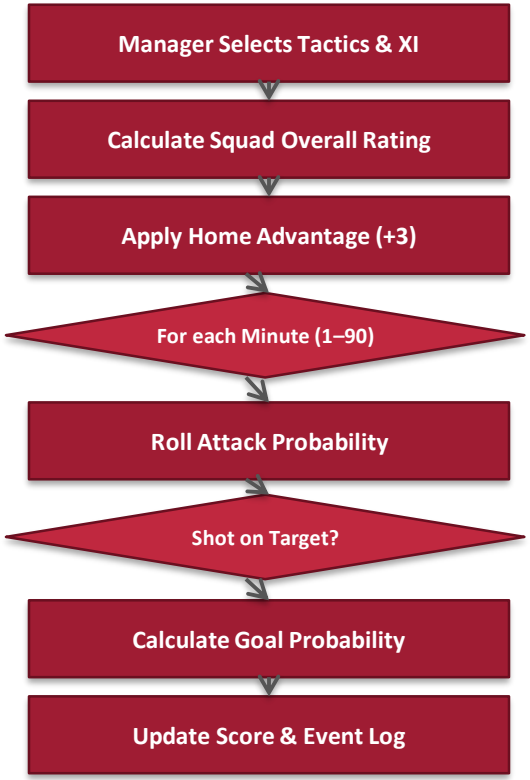
SQLAlchemy ORM
Player / Club / Match / Transfer Models

Match Engine
Probability-based score simulator

Data Seeder
5 leagues · 100+ clubs · Real player stats

SQLite DB
Persistent storage for season data

MATCH SIMULATION FLOW



No goal → Chance event logged
Loop exits after minute 90

D Y PATIL DEEMED TO BE UNIVERSITY — RAMRAO ADIK INSTITUTE OF TECHNOLOGY NAVI MUMBAI

Football Manager — Technology Stack

Technology	Version	Purpose / Role
Python 3.12	3.12	Core backend language for all game logic and models
Flask	3.x	Lightweight web framework for routing, sessions, and templates
SQLAlchemy	2.x	ORM for Player, Club, Match, Transfer, Season models
SQLite	3.x	Embedded relational database for persistent season storage
Jinja2	3.x	HTML templating engine for dynamic web pages
JSON Data Files	—	Real league data: Premier League, La Liga, Bundesliga, Serie A, Ligue 1
HTML / CSS / JS	—	Frontend UI for manager dashboard and match viewer
Python random	stdlib	Probabilistic match simulation (goal and chance events)

5
Leagues

100+
Clubs

2000+
Players

6
AI Techniques

D Y PATIL DEEMED TO BE UNIVERSITY — RAMRAO ADIK INSTITUTE OF TECHNOLOGY NAVI MUMBAI

Football Manager — Key Features & AI Techniques

Feature	AI Method Used	Description
Attribute-Based Match Simulation	Weighted Probability Model	Squad overall rating → attack & goal probabilities computed per minute
Home Advantage Modelling	Domain-Specific Heuristic	Home club receives +3 overall bonus before probability normalization
Tactics Impact Engine	Rule-Based Expert System	Formation + Mentality + Pressing style modify effective team strength
Transfer Market AI	Constraint Satisfaction	Release clause, wage cap, and contract rules govern legal transfers
Player Morale System	State Machine	Morale (1–100) changes after wins/losses and affects match performance
Season Simulator	Monte Carlo Loop	90-minute event loops iterated across a full fixture list
League Table Generator	Sorting Algorithm	Points → GD → GF to rank all clubs at any fixture round

D Y PATIL DEEMED TO BE UNIVERSITY RAMRAO ADIK INSTITUTE OF TECHNOLOGY NAVI MUMBAI

Epic Battle Simulation — Problem Statement

Static AI opponents: Most RTS/war games use scripted, predictable AI that does not learn from player behavior.

No adaptive strategy: AI armies follow fixed composition rules regardless of how the player adapts mid-battle.

Expensive RL frameworks: Reinforcement learning in games often requires TensorFlow/PyTorch — prohibitively heavy for indie/academic projects.

3D spatial intelligence missing: Existing academic RL demos are grid-world only; none handle real 3D spatial pathfinding with terrain.

No persistent memory: Battle AI resets every game — it cannot retain learned strategies across sessions.

D Y PATIL DEEMED TO BE UNIVERSITY — RAMRAO ADIK — INSTITUTE OF TECHNOLOGY NAVI MUMBAI

Epic Battle Simulation — Objectives

1. Build a real-time 3D war game using Panda3D with Soldiers and Tanks as controllable unit types.
2. Implement tabular Q-learning AI that selects from 6 strategic actions based on battlefield state.
3. Encode state as a 4-tuple: (strength_bucket, tank_bucket, map_control, distance_bucket).
4. Support 6 AI actions: spawn_more_soldiers, spawn_more_tanks, balanced_army, rush_attack, defend, spread_units.
5. Use epsilon-greedy exploration (ε=0.2) with α=0.5 learning rate for stable, fast convergence.
6. Persist Q-table to JSON (ai_memory.json) so the AI retains learning across multiple game sessions.
7. Implement spatial grid acceleration for O(1) nearest-enemy lookups in large-scale 50,000-point battles.

D Y PATIL DEEMED TO BE UNIVERSITY RAMRAO ADIK INSTITUTE OF TECHNOLOGY NAVI MUMBAI

Epic Battle Simulation — System Architecture & Game Loop

SYSTEM LAYERS

Panda3D Engine
3D rendering, task manager, input

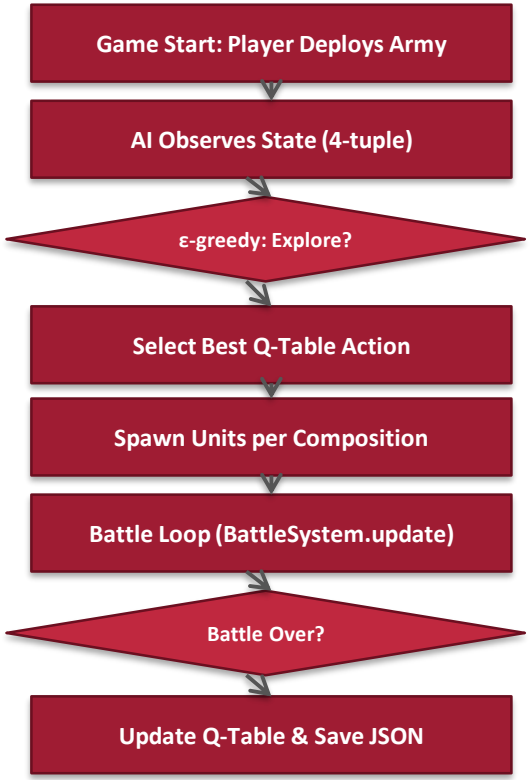
BattleSystem
Movement, combat, victory conditions

AITrainer (Q-Learning)
State encoding, action selection

SpatialGrid
O(1) nearest-enemy lookup

JSON Q-Table Store
Persistent cross-session memory

Q-LEARNING GAME LOOP



D Y PATIL DEEMED TO BE UNIVERSITY — RAMRAO ADIK INSTITUTE OF TECHNOLOGY NAVI MUMBAI

Epic Battle Simulation — Technology Stack

Technology	Version	Purpose / Role
Python 3.12	3.12	Core language for all game logic, AI, and simulation
Panda3D	1.10.x	3D game engine: rendering, scene graph, task manager, physics
Tabular Q-Learning	Custom	Reinforcement learning — no external ML library required
JSON (ai_memory.json)	stdlib	Persistent Q-table storage across game sessions
SpatialGrid	Custom	O(1) cell-partitioned grid for fast nearest-enemy targeting
Python random / math	stdlib	Probabilistic action selection (epsilon-greedy)
Vec3 (Panda3D core)	1.10.x	3D vector maths for unit movement and position tracking
Panda3D CollisionSystem	1.10.x	Terrain detection and unit collision resolution

6
AI Actions

4
State Dims

0.5
α (LR)

0.2
ε (Explore)

D Y PATIL DEEMED TO BE UNIVERSITY — RAMRAO ADIK INSTITUTE OF TECHNOLOGY NAVI MUMBAI

Epic Battle Simulation — Key Features & AI Techniques

Feature	AI Method Used	Description
Q-Learning Strategy Selector	Tabular Reinforcement Learning	6-action Q-table updated via Bellman equation after each battle
Epsilon-Greedy Exploration	Exploration vs Exploitation	ε=0.2 ensures the AI tries new strategies while exploiting known wins
State Space Encoding	Feature Engineering	4-tuple: strength ratio, tank density, map control, distance bucket
Army Composition Planner	Rule-Based Composition from Q-Action	Maps high-level action to soldiers:tanks ratio and tactical style
Spatial Grid Targeting	Geometric Hash Grid	O(1) nearest-enemy lookup across 50,000-unit armies
Terrain-Aware Movement	Context-Sensitive Pathfinding	Units adjust speed and attack range based on terrain type
Cross-Session AI Memory	JSON Q-Table Persistence	AI improves across separate game sessions via saved Q-values

D Y PATIL DEEMED TO BE UNIVERSITY — RAMRAO ADIK — INSTITUTE OF TECHNOLOGY NAVI MUMBAI

Minecraft AI Builder — Problem Statement

NLP-to-3D gap: No existing open-source tool converts free-form natural language directly into live 3D block structures.

API dependency: Existing AI builders (ChatGPT plugins, Midjourney) require internet APIs — unusable offline or in academic settings.

No compound prompt support: Users cannot say 'build a castle next to a bridge' — no system handles multi-structure decomposition.

Heavy ML stacks: Block-world AI projects use transformer models (GPT-2, BERT) that are too resource-intensive for local execution.

Static generation: Existing procedural generators place all blocks instantly; no live animated placement exists.

Minecraft AI Builder — Objectives

1. Train TF-IDF + Logistic Regression classifiers on 30,000+ synthetic examples for intent, size, and material prediction.
2. Implement CompoundSplitter to parse multi-structure commands (e.g. 'a castle and a bridge') into sub-prompts.
3. Achieve >95% average accuracy across intent, size, and material classifiers on held-out test data.
4. Build 8 distinct structure types procedurally: castle, house, tower, bridge, pyramid, dome, wall, village.
5. Animate block placement live at ~50 blocks/second using Panda3D's task scheduler.
6. Support drag-and-drop repositioning: click to remove, hold to move whole structures by structure_id.
7. Add an animated Companion NPC that walks around the world independently of the build system.
8. Keep entire pipeline offline — no internet dependency for training, inference, or runtime execution.

Minecraft AI Builder — NLP Pipeline Flowchart

SYSTEM MODULES

CompoundSplitter
Detects & splits multi-structure prompts

TrainedParser
TF-IDF vectoriser + Logistic Regression

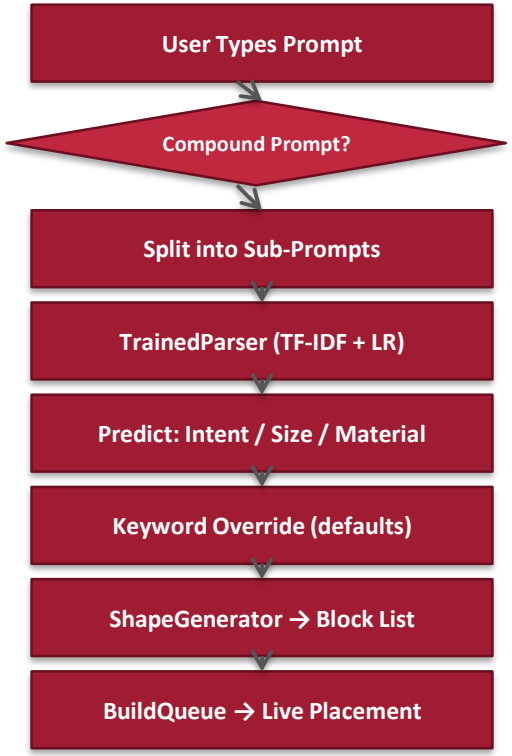
KeywordOverrider
Style & material default fallbacks

ShapeGenerator
8 procedural structure algorithms

BuildQueue
Animated live placement at 50 blk/s

DragDropSystem
structure_id grouped move/remove

NLP → 3D PIPELINE



Single prompt skips split

Structure Built!

D Y PATIL UNIVERSITY — RAMRAO ADIK INSTITUTE OF TECHNOLOGY NAVI MUMBAI

Minecraft AI Builder — ML / AI Pipeline

Classifier	Output Classes	Test Accuracy
Intent Classifier	castle, house, tower, bridge, pyramid, dome, wall, village	>97%
Size Classifier	small, medium, large, huge	>95%
Material Classifier	stone, wood, brick, glass, sand, dirt	>94%

TRAINING PIPELINE

Step 1: Data Generation

30,000+ synthetic prompts
Programmatically varied phrasing

Step 2: Vectorisation

TF-IDF: ngram (1,2)
max_features = 5,000

Step 3: Training

Logistic Regression
L-BFGS solver, 85/15 split

Step 4: Serialisation

joblib pickle export
Loaded at runtime, zero latency

30K+
Training Examples

>95%
Avg Accuracy

3
Classifiers

8
Structure Types

D Y PATIL DEEMED TO BE UNIVERSITY RAMRAO ADIK INSTITUTE OF TECHNOLOGY NAVI MUMBAI

Minecraft AI Builder — Technology Stack

Technology	Version	Purpose / Role
Python 3.13	3.13	Core language for NLP pipeline, game logic, and 3D engine integration
Panda3D	1.10.x	3D rendering engine: scene graph, task scheduler, collision system
scikit-learn	1.5.x	TF-IDF vectoriser and Logistic Regression classifiers
TF-IDF Vectoriser	sklearn	Text feature extraction with ngram_range=(1,2), max_features=5000
Logistic Regression	sklearn	Multi-class classifier with L-BFGS solver for Intent / Size / Material
joblib	sklearn	Model serialisation (pickle) for fast load at runtime
GLB / 3D Models	glTF 2.0	3D block and NPC mesh files rendered via Panda3D loader
PowerShell InputBox	Windows	Native GUI for prompt input (Windows-only — identified as a limitation)

~50
Blocks/sec

0ms
API Latency

3.13
Python

<1s
Inference

D Y PATIL DEEMED TO BE UNIVERSITY — RAMRAO ADIK INSTITUTE OF TECHNOLOGY NAVI MUMBAI

Minecraft AI Builder — Features & Supported Structures

Structure	Algorithm Summary	~Block Count
Castle	Hollow rectangle walls + 4 corner towers + gate archway	~200–600
House	4-wall box + sloped/flat roof + door placement	~80–250
Tower	Cylindrical or rectangular vertical shaft + top platform	~40–150
Bridge	Flat span + railing columns, spans two terrain points	~60–180
Pyramid	Square-base stepped reduction, layer by layer	~100–500
Dome	Sphere-approx algorithm, filled or hollow shell	~120–400
Wall	Long flat barrier with optional battlements on top	~30–300
Village	Cluster of 3–6 small houses + path connectors	~300–900

D Y PATIL DEEMED TO BE UNIVERSITY — RAMRAO ADIK — INSTITUTE OF TECHNOLOGY NAVI MUMBAI

Comparative Overview — All Three Games

Metric	⚽ Football Manager	⚔️ Epic Battle Simulation	⛏️ Minecraft AI Builder
AI Approach	Weighted Probability Model	Tabular Q-Learning (RL)	TF-IDF + Logistic Regression (ML)
Language	Python 3.12	Python 3.12	Python 3.13
Framework	Flask + SQLAlchemy	Panda3D	Panda3D + scikit-learn
Dataset Type	Real league JSON + SQLite	Self-generated Q-table (JSON)	30K+ synthetic NLP prompts
Output Type	Match events + League Table	3D battle simulation	Live 3D block structures
Complexity	Web app + DB + match engine	3D engine + RL agent + Spatial Grid	NLP pipeline + 3D engine + ML models
Platform	Browser (Flask web)	Desktop 3D (Panda3D)	Desktop 3D (Panda3D)

D Y PATIL DEEMED TO BE UNIVERSITY RAMRAO ADIK INSTITUTE OF TECHNOLOGY NAVI MUMBAI

Advantages — All Three Games

⚽ Football Manager

✓ Real league data — 5 leagues, 100+ clubs seeded automatically

✓ Attribute-driven AI — not random; player stats drive outcomes

✓ Full-stack — Flask web app covers squad, tactics, transfers, season

✓ Offline & free — no subscriptions, no API keys needed

⚔ Epic Battle Simulation

✓ Cross-session memory — AI improves with every battle played

✓ No ML libraries — pure Python Q-learning in a 3D game engine

✓ Spatial Grid — O(1) targeting scales to 50,000+ units

✓ 6 adaptive actions — AI selects rush, defend, spread, or balance

Minecraft AI Builder

✓ >95% accuracy — 3 separate classifiers, 30K+ training examples

✓ Compound prompts — 'castle and bridge' handled in one command

✓ Fully offline — zero latency, no internet, no API dependency

✓ Live animation — blocks placed one-by-one at ~50 blocks/sec

D Y PATIL DEEMED TO BE UNIVERSITY RAMRAO ADIK INSTITUTE OF TECHNOLOGY NAVI MUMBAI

Limitations & Future Scope — All Three Games

Game	Current Limitation	Future Scope	Priority
⚽ FM	No real-time match replay — event log only	3D match visualisation with player positions	Medium
⚽ FM	Transfer AI uses simple constraints — no bidding war	Auction-based transfer market with multiple clubs	Low
⚽ FM	No training simulation — players don't improve in-game	Weekly training sessions affecting player growth	High
⚔️ Battle	Q-table grows large with many state combinations	Deep Q-Network (DQN) for continuous state space	High
⚔️ Battle	Only 2 unit types: Soldier and Tank	Artillery, Air units, Supply chains	Medium
⚔️ Battle	No multi-player mode — single human vs AI only	LAN multiplayer via Python sockets	Low
⛏️ MC	Windows-only — PowerShell InputBox limits portability	Cross-platform GUI (tkinter / PyQt)	High
⛏️ MC	No save/load — builds lost on close	JSON / SQLite world persistence	High
⛏️ MC	Only 6 block types — no water, glass, or ore	Full block palette with physics simulation	Medium

D Y PATIL DEEMED TO BE UNIVERSITY RAMRAO ADIK INSTITUTE OF TECHNOLOGY NAVI MUMBAI

NexusPlay — Unified Game Portal

What is NexusPlay?

A local Flask web server (port 8000) that acts as a single dashboard to browse and launch all three AI-powered games from one browser tab — no manual terminal commands required.

Website Sections

Home Hero landing with CTA button
About Project description and purpose
Architecture System diagram and tech explanation
Features Per-game AI feature highlights
Applications Game cards with one-click Launch buttons
Tech Stack Technology tags for the whole portal

Portal Tech Stack

Flask 3.x — web server on port 8000
Jinja2 — server-side game card templating
HTML5 / CSS3 — Glassmorphism dark UI
JavaScript ES6+ Fetch API — async launch
flask-cors — cross-origin subprocess support
webbrowser module — auto-opens on startup

3	8000	1-Click
Games Integrated	Portal Port	Game Launch

Website Integration — Architecture & File Structure

NEXUSPLAY / GAME /

```
portal/
  app.py            (Flask orchestrator)
  templates/index.html  (Jinja2 UI)
  static/script.js      (Fetch API calls)
  static/style.css      (Glassmorphism)
  static/images/       (game thumbnails)
main.py               (Epic Battle game)
  battle_system.py
  ai_learning.py  |  units.py
  ai_memory.json  |  models/
fm sim/fm sim 1/fm sim/
  app.py  |  engine/  |  routes/
  templates/  |  static/  |  data/
minecarft_ai_project/
  main.py  |  ai_builder.py
  model.pkl  |  *.glb  |  skybox/
```

Integration Flow

User opens Browser
http://127.0.0.1:8000

GET / (renders index.html via Jinja2)

Flask Orchestrator (portal/app.py)
GAMES dict maps game_id to script path

POST /api/launch/game_id (JS Fetch)

subprocess.Popen(python script)
New console window, correct cwd

spawns detached game process

Panda3D Battle Sim	**Flask** FM Sim :5000	**Panda3D** MC AI Builder

Per-Game Launch Integration — How Each Game Runs

⚔ Epic Battle Simulator

game_id: battle_sim
Entry point:
`GAME/main.py`
Engine: Panda3D 3D
Key files loaded:
`battle_system.py`
`ai_learning.py`
`units.py`
`ai_memory.json`
`models/*.glb`
Launch mode:
New console window (CREATE_NEW_CONSOLE). Opens Panda3D window directly.

⚽ Football Manager Sim

game_id: fm_sim
Entry point:
`fm sim/fm sim 1/fm sim/app.py`
Engine: Flask + SQLAlchemy
Key folders loaded:
`engine/`
`routes/`
`models/`
`templates/  data/`
Launch mode:
Subprocess starts Flask on port 5000. Portal JS auto-opens http://127.0.0.1:5000 in new tab after 2s.

⛏ Minecraft AI Builder

game_id: minecraft_ai
Entry point:
`minecarft_ai_project/main.py`
Engine: Panda3D + scikit-learn
Key files loaded:
`ai_builder.py`
`model.pkl  (joblib classifiers)`
`*.glb  (block & NPC meshes)`
`skybox/  companion.py`
Launch mode:
New console window. Loads model.pkl at startup, opens Panda3D 3D world with NLP prompt input.

Conclusion

⚽ **Football Manager**	A full-stack web football management game with attribute-driven match simulation across 5 real leagues, complete with transfers, morale, tactics, and season progression.
⚔️ **Epic Battle Simulation**	A 3D real-time battle game with a tabular Q-learning AI that adapts army composition per battle, retains learning across sessions, and handles thousands of units via spatial grids.
❓ **Minecraft AI Builder**	An offline NLP-to-3D system where TF-IDF + Logistic Regression classifiers (>95% accuracy, 30K+ examples) convert natural language into live animated 3D block structures.

30K+	3	8	5	>95%
Training Examples	AI Paradigms	Structure Types	Real Leagues	ML Accuracy

D Y PATIL DEEMED TO BE UNIVERSITY — RAMRAO ADIK INSTITUTE OF TECHNOLOGY NAVI MUMBAI

References

[1] Python 3.13 Official Documentation — https://docs.python.org/3/

[2] Flask Web Framework Documentation — https://flask.palletsprojects.com/

[3] SQLAlchemy ORM Documentation — https://docs.sqlalchemy.org/

[4] Panda3D Game Engine Manual and API Reference — https://docs.panda3d.org/

[5] scikit-learn User Guide — TF-IDF & Logistic Regression — https://scikit-learn.org/stable/

[6] Sutton & Barto — Reinforcement Learning: An Introduction (2nd Ed.) — MIT Press, 2018

[7] TF-IDF Feature Extraction — sklearn.feature_extraction.text.TfidfVectorizer

[8] glTF / GLB 3D Model Format Specification — https://registry.khronos.org/glTF/

[9] joblib Serialisation Library — https://joblib.readthedocs.io/

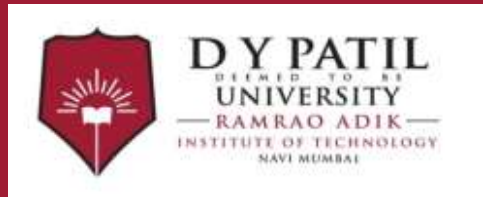
[10] Watkins & Dayan — Q-Learning. Machine Learning, 8(3–4), 279–292, 1992

[11] NumPy Documentation — https://numpy.org/doc/

[12] Project Source Code — Football Manager, AI War Simulation, Minecraft AI Builder

D Y PATIL UNIVERSITY — RAMRAO ADIK — INSTITUTE OF TECHNOLOGY NAVI MUMBAI

Thank You


D Y PATIL
DEEMED TO BE
UNIVERSITY
RAMRAO ADIK
INSTITUTE OF TECHNOLOGY
NAVI MUMBAI